

Spec-Driven Development на практике: спеки, контекст и работа с агентами

TL;DR. Spec-Driven Development (SDD) — это набор инженерных практик, помогающий обойти фундаментальное ограничение LLM: окно контекста. Ключевая идея — концентрировать «essential»-знание в короткой спецификации и подавать её в самое начало сессии агента, чтобы тот работал в пределах ~100 тыс. полезных токенов. Спеки бывают трёх уровней (история / эпик / проект), их не стоит хранить вечнозелёными, а простейшая реализация SDD — это plan-mode, которым многие уже пользуются, не зная названия.

Почему вообще нужен SDD: проблема контекста

LLM имеет Context Window и, отдельно, Usable Context Window — реально пригодную для работы часть, которая много меньше заявленного размера. Заявленный «1 миллион токенов» у моделей не означает, что до миллиона всё работает: модель начинает «плыть» гораздо раньше. По словам Алексея, при ~500 тыс. токенов вероятность вспомнить нужный токен из середины сессии падает примерно до 80% — для брейнсторминга это нормально (люди тоже забывают, что было полчаса/~50 тыс. токенов назад), но для написания кода критично, потому что именно так появляются failure modes. Автоматическая компрессия контекста («компакт») — это тот же lost-in-the-middle: модель выкидывает токены, которые ей показались неважными, а она плохо понимает, что важно. Вывод: цифру токенов надо мониторить на каждом ходе и, перевалив за ~100 тыс., останавливать сессию и начинать новую. Usable-контекст за последний год вырос не в 5–10 раз, а процентов на 20 (ссылка на статьи Anthropic про Context Rot), и он коррелирует с размером модели, где уже упёрлись в ограничения по железу.

Суть SDD и «идеальная сессия Васи»

Метафора Алексея: агент каждый раз «просыпается» перед белой стеной, где написано «тебя зовут Вася, вот твоя задача». Идеальная кодировочная сессия — когда в самое начало заливается концентрированный контекст (спека, ~1500 токенов), после которого всё ясно и не надо никуда ходить искать. Если вместо этого «Петя» по чайной ложке рассказывает, что ему надо, две трети полезного контекста уйдёт просто на выяснение намерения, а на код/проверку/коммит уже не хватит. Поэтому спеку пишут в отдельной planning-сессии. Егор сформулировал закрепляющий тезис: в рамках подкаста SDD — это в первую очередь спецификация как артефакт, задача которого передать агенту в текущей сессии нужный контекст, чтобы он справился лучше. Идея «спека как единственный источник истины, из которого 10 раз в секунду регенерируется весь код» (OpenSpec-подход) — красивая, но, по мнению Алексея, опережает технологии: пока работает плохо, источник истины — сам код (с комментариями «почему»). Plan-mode — это самая примитивная реализация SDD; отличие спеки в том, что она высечена в камне и лежит рядом с проектом, а план живёт только внутри одного чата.

Три уровня спецификаций¹

¹Терминология не устоялась; Алексей мыслит в триаде Project → Epic → Story. Размер спеки (1500 токенов) — это грубый механизм, заставляющий модель-автора выкинуть неважное.

Уровень	Масштаб	Оптимальный размер	Роль и особенности
Story (одна coding session)	1 задача, ~100–300 строк кода	intent ~300 токенов (≈полстраницы); вся спека ~1500 токенов (≈3 страницы)	Заточена строго на кодинг-агента. Содержит intent, Input/Output Matrix (правила вход→выход, из которых потом делаются тесты), acceptance criteria, список того, что делать НЕ надо, non-functional requirements (только если ими прицельно занимаемся), указания какие файлы/куски прочитать перед стартом
Epic	5–10 историй = цельная фича		Все acceptance-критерии выписываются и распределяются по историям; важна читаемость для человека; уже работают IF/WHEN-формулировки и диаграммы
Project	40–100 историй / ~10 эпиков, работа на неделю+	~20–40 «guiding thoughts» (коротких параграфов)	Контекст на вход планировщику эпиков; заточена и на человека (диаграммы, стрелочки-квадратики); задаёт последовательность и параллелизацию (граф зависимостей эпиков). Из «guiding thoughts» можно сгенерировать любое представление (PRD и т.п.) по требованию

Что писать в спеку истории (и что не писать)

- Intent (~300 токенов): что хочет человек. Если не влезает в полстраницы — это уже больше одной истории.
- Продуктовые требования живут в intent, а не отдельно.
- Input/Output Matrix: набор правил «если на вход прилетело X — на выходе Y», покрывающих domain boundaries (напр.: 0 пересадок → все показанные перелёты 0 пересадок; что делать если нашлось 0; что делать если >20). Это прямо мапится в код и тесты.
- Acceptance criteria, которые нельзя выразить через матрицу.
- Список вещей, которые агент захочет сделать, но делать НЕ надо (борьба с фантазированием LLM). Проблема: список бесконечен, но LLM-сгенерированный список «летающих тарелок» коррелирует с тем, что LLM реально начал бы делать.
- Абстрактные правила + пара примеров, если правило хоть немного сложное (правильно/нет — Алексей честно не знает).
- Non-functional requirements — только если они играют роль именно в этой истории; иначе агент начнёт их мерить, а это дорого.
- Ключевой принцип (формулировка Егора): в спеку пишем то, что НЕ хотим, чтобы агент додумал сам. Как именно писать код (for/while) — пусть решает агент из своего контекста.
- Планировщику не надо принимать решения, которые может принять кодинг-агент из своего контекста (этим грешит plan-mode, кодируя прямо в Markdown).

Процесс: трассирующие пули, human-in-the-loop и откаты

Внутри эпика неопределённость и решения по внутреннему дизайну сдвигают «влево» — в первую-вторую истории, которые исполняют в режиме human-in-the-loop (это, по сути, дизайн-ревью). Первая история часто = tracer bullet: прострелить фичу через все слои (UI, backend, persistence, audit) с простейшим юзкейсом, после чего представление о том, как делать, радикально меняется — примерно в половине случаев. Остальные истории эпика можно запускать оркестратором почти без участия человека, а ревью делать целиком в конце.

Важный сдвиг мышления: если результат не нравится, часто продуктивнее не патчить 1000 строк, а понять, где вылезла ошибка (в дизайне/спеке или в коде→спеке), откатиться туда, поправить спеку и регенерировать код за полчаса — потому что теперь это дёшево (агент в ~100 раз дешевле ручной работы). Это НЕ waterfall: сейчас дёшево принимать часть решений заранее (в отличие от 90-х, когда правка UML-диаграмм руками съедала полжизни), но не надо и уходить в другую крайность. Комментарии в коде «перестали гнить» — LLM их поддерживают и находят, поэтому старые спеки Алексей архивирует, а не держит вечно-зелёными.

Когда SDD не нужен и границы применимости

- Тривиальные задачи (blast radius = 0, поправить 3 строки в одном файле) решаются без SDD; в BMAD для этого есть скилл QuickDev. Ревью при этом всё равно нужно (хотя бы один слой).
- Если все 4 стадии (intent discovery → сбор контекста по коду → планирование → написание кода) влезают в одну сессию ~50–100 тыс. токенов — SDD почти ничего не даёт. Чем хуже влезает, тем больше даёт.
- UI/пиксели — слабое место всех SDD-подходов. Кодинг-агент не должен смотреть на скриншоты (дорого по токенам). Два лагеря: (1) сделать pixel-perfect дизайн-систему с компонентами и оперировать компонентами в спеке; (2) «наплевать на красоту». Рабочий приём — отдавать агенту не пиксели, а структуру: Figma MCP (вложенные контейнеры, констрейнты) или экспорт в HTML (агенты хорошо генерят UI-код из HTML).
- Хакатон/маленький проект — плохо ложится на тяжёлый «микрорайонный» процесс BMAD ранних версий (Стас: два часа отвечал на 300 вопросов, из которых по делу два). Инструмент был не тот; сейчас BMAD лучше подходит под мелкие задачи, если знать какими скиллами пользоваться.
- Формальная верификация — только для очень простых доменов или критичных систем (ядерный реактор, станок с резцом); для ~90% софта слишком дорого и разваливается на сложности.

Мифы и практические ответы

Тезис / миф	Ответ Алексея
Смыслы теряются: язык лосси, агент делает не то	Проблема не уникальна для SDD; SDD её не решает. Спасает многослойная обратная связь: автотесты → agentic review → human review → ... → продакшн
Зелёные тесты гарантируют корректность	Переоценены. LLM пишет тесты по тем же весам, что и код — бывает 100% coverage, CI зелёный, а код неправильный. Автотесты фронтенда выше smoke-уровня — неблагодарное дело
SDD жрёт много токенов и это дорого	Алексей гоняет Opus на high reasoning; двух подписок по \$100 (Claude + Codex) обычно хватает. Полная цена по API была бы ~20–30% зарплаты — окупается ускорением хотя бы в 1.5×. Для больших корпораций (нет подписок, платят по API) — оптимизировать реально стоит уже сейчас
SDD только для соло-разработчиков, в командах не заводится	Заводится (много контрпримеров). Спеку истории пишет отдельный разработчик; распределение работы — по эпикам (эпик ~1 день), с графом зависимостей; специализации команд обычно живут в разных репо/директориях
Роль системного промпта «ты гениальный фронтендер»	Перестало работать ~год назад. Специализация работает только через разный контекст для разных сессий, а не через ролевые ярлыки
SDD ещё почти никто не использует	Иллюзия: половина работающих с Claude Code уже делает SDD через plan-mode и slash-review, просто не зная названия. Идеи из BMAD и др. через 2–3 месяца попадают в харнесы (Claude Code и т.п.)

Умрёт ли SDD, когда модели поумнеют

Идеальный SDD-фреймворк = один промпт «спроси у человека, что надо, и сделай». Но для этого нужен идеальный агент с почти бесконечным usable-контекстом, идеальным recall и пониманием essential complexity (Фред Брукс, «No Silver Bullet»). Алексей такого не видит: растёт точность исполнения, а не понимание важного. Его аналогия — управляемый термоядерный синтез: «за углом» уже 80 лет; с AGI, вероятно, та же история. При этом польза здесь и сейчас есть — прирост производительности «раза в два с половиной» на его репозитории при переходе от «ругаться и плакать» к методу.

Бенчмарки и эвалы Алексей считает малополезными: они меряют синтетику и микрозадачи, а не дописывание LLM-кода в большую живую кодобазу; оверфитинг ради 5% на SWE — это «полировать палку, чтобы копать ей как лопатой». Критерий истины — практика на собственных понятных примерах и, для опенсорса, быстрый фидбэк сообщества.

Открытые вопросы

- Где именно проходит граница «примеры vs абстрактные правила» в спеке — сам Алексей признаёт, что не знает правильного ответа.
- Как SDD работает в крупных распределённых организациях с множеством микросервисов — Алексей говорит только со слов других и предполагает, что где-то не работает.
- Нет публичного ресёрча/эвала именно по SDD (только по plan-mode и context rot по касательной); объективно измерить пользу пока нечем.
- Как поддерживать спеку в синхроне с кодом при фиксе сложных багов (поднято комментатором @otetz77) — процесс болезненный и на практике будет редким.
- Конкретные фреймворки (BMAD builder, Spres-Kit, OpenSpres, supergrowers) и их сравнение в выпуске сознательно не разбирались.
- Как в SDD встроить типы задач, которые разработчик в принципе не готов/не может делегировать агенту (30% ручного кода).

Моё мнение

Согласен с центральным тезисом: usable-контекст — реальное узкое место, и практика «короткая спека в начало чистой сессии + мониторинг токенов + рестарт при ~100k» объективно работает, это не каргокульт (комментатор @TheSerenity4444 удачно сжал суть: SDD отличается от промптинга тем, что спека — это эталон-валидатор результата с очерченным DoD). Особенно ценна мысль про откат-и-регенерацию вместо патчинга и про Input/Output Matrix как мост к тестам. Частично не согласен с лёгким отношением к стоимости токенов: сам гость признаёт, что для корпораций без подписок экономика другая, а комментаторы (@Олег, @AlekseyCounter) справедливо замечают, что мы рискуем сделать разработку дороже и медленнее, добавив слой валидации — это стоит честно закладывать в ROI, а не отмахиваться «жизнь коротка». Тезис «автотесты фронтенда выше smoke — неблагодарное дело» я считаю спорным и уже устаревающим: современные component/visual-regression инструменты вполне окупаются. Скепсис к бенчмаркам разделяю, но вывод «мерить только чуйкой на своих примерах» опасен — Стас прав, что так плодятся каргоскиллы; золотая середина — лёгкий внутренний A/B на реальных задачах команды с разбором failure modes и token consumption. От себя добавил бы: (1) держать в репозитории не сами спеки-истории, а шаблон + чек-лист «что НЕ додумывать», раз спеки одноразовые; (2) для UI не выбирать между двумя лагерями, а комбинировать — дизайн-система для 80% рутины плюс HTML/Figma-структура как контекст для нестандартного; (3) явно логировать в спеке, какие acceptance-критерии закрывает каждая история, чтобы граница эпика была проверяемой. И, соглашаясь с комментатором @romanivanov6183 в шутке про «сжать в два предложения», отмечу иронию: подкаст о концентрации смысла сам получился очень разреженным — but соге-идея того стоит.